DISHCOVERY – YOUR COOKING ASSISTANT

A Smart AI-Based Recipe Recommendation System

Submitted by:
Tanisha Majmundar

[202307020117]

Riaan Sendhav
[202307020095]

Vanshika Mehta

[202307020122]

Team ID: CU_CP_Team_9434

Under the guidance of:
Mr. Palwinder Singh

Course: BTech AIML

INTRODUCTION

In today's fast-paced lifestyle, one of the most recurring challenges individuals face is deciding what to cook, especially when they are constrained by the limited ingredients available at home. Whether it is students living away from home, working professionals managing tight schedules, or beginners in the kitchen, the dilemma of "what to cook" persists across all groups.

Traditional methods of recipe discovery—browsing cookbooks, watching long cooking videos, or endlessly scrolling through recipe websites—are often time-consuming, lack personalization, and can result in repetitive meals or food wastage. These approaches also fail to take into account the user's real-time kitchen inventory, dietary restrictions, and available cooking time.

To address this everyday problem with the help of modern technology, we have developed **Dishcovery – Your Cooking Assistant**, an AI-powered web-based application. Dishcovery intelligently suggests recipes based on ingredients provided by the user. It is designed to be user-friendly, efficient, and scalable. With features such as machine learning-powered recipe matching, search and filter capabilities, dynamic recipe cards, and data-driven analytics, Dishcovery aims to revolutionize the way people cook at home.

The goal of the project is not just to offer recipe suggestions, but to create an interactive ecosystem that minimizes food wastage, simplifies meal planning, and encourages experimentation in the kitchen using the ingredients users already have.

Dishcovery merges Artificial Intelligence with real-world cooking needs, offering users a smart and personalized experience that makes the task of cooking enjoyable and less stressful.

PROBLEM STATEMENT

Cooking is a fundamental part of daily life, yet deciding *what to cook* remains a repetitive challenge for many. The problem becomes more complicated when individuals have only a limited set of ingredients at hand. This situation is common among students, working professionals, and home cooks, who often find themselves short on time, ingredients, or inspiration.

The traditional process of finding a recipe involves:

- Browsing through numerous online cooking blogs or apps,
- Filtering out recipes that don't match the available ingredients,
- Spending time reading lengthy preparation steps,
- And dealing with suggestions that may not align with their dietary preferences or cooking skills.

This approach is not only time-consuming but also inefficient. It leads to:

- **Decision fatigue**, as users are overwhelmed with too many choices,
- **Repetitive meals**, due to lack of variety,
- **Food wastage**, as unused ingredients are forgotten or discarded,
- And **missed opportunities**, where simple dishes could have been made with what was already available.

To solve this problem, there is a growing need for an intelligent, user-focused solution that:

- Accepts the ingredients currently available to a user,
- Understands dietary restrictions and preferences,
- Considers the user's cooking time and skill level,
- And delivers quick, reliable, and relevant recipe suggestions.

Our project, **Dishcovery – Your Cooking Assistant**, addresses these challenges using **Artificial Intelligence** and **Web Technologies**. By creating a smart system that provides personalized recipe recommendations in seconds, we aim to make home cooking simpler, more efficient, and less stressful.

OBJECTIVES

The primary objective of this project is to develop a smart and intuitive recipe recommendation system that empowers users to cook meals using ingredients they already have, while enhancing their overall cooking experience through personalized, AI-driven suggestions.

To achieve this, the following specific objectives were defined:

1. **Develop a Web-Based Application:**
    - Create a responsive, interactive, and user-friendly web platform using modern web technologies.
2. **Enable Ingredient-Based Recipe Suggestions:**
    - Allow users to input the ingredients they currently have at home.
    - Use machine learning techniques to match inputs with suitable recipes.
3. **Integrate Machine Learning Algorithms:**
    - Utilize **CountVectorizer** to convert text-based ingredients into numerical vectors.
    - Apply the **K-Nearest Neighbors (KNN)** algorithm to find and recommend the most relevant recipes based on ingredient similarity.
4. **Design an Intuitive User Interface:**
    - Display recipes in the form of well-structured recipe cards.
    - Include preparation steps, cooking instructions, tips, cooking time, dish type, and visuals where available.
5. **Implement Search and Filtering Features:**
    - Enable users to explore recipes by name or filter by dish type (e.g., breakfast, dessert, etc.).
    - Improve discoverability and ease of use.
6. **Build an Admin and Analytics Dashboard:**
    - Provide admin users the ability to manage recipe data and view user interaction statistics.
    - Generate visual insights (e.g., popular cuisines, most searched ingredients) using data analytics libraries.

These objectives collectively aim to create a platform that bridges technology with everyday cooking, reducing the burden of meal planning and promoting better use of food resources.

LITERATURE REVIEW / EXISTING SYSTEMS

In recent years, digital platforms have revolutionized how people access and engage with culinary content. Numerous recipe apps and websites exist that provide users with access to thousands of dishes across various cuisines. However, despite their popularity, many of these platforms fall short in one critical area — personalization based on **available ingredients** and **real-time constraints**.

Below are a few widely used systems and their limitations:

---

1. Tasty (Buzzfeed)

- **Overview:** Offers thousands of recipes with short videos and step-by-step instructions.
- **Strengths:** Visually engaging, easy to follow, and beginner-friendly.
- **Limitations:** Lacks a feature to input ingredients and get suggestions based on what's currently available in a user's kitchen.

---

2. AllRecipes

- **Overview:** A popular online recipe repository with user reviews, ratings, and meal planners.
- **Strengths:** Advanced filtering options and a large recipe database.
- **Limitations:** The "ingredient search" feature is limited in accuracy and lacks AI-powered matching. It doesn't rank recipes by similarity or cooking time dynamically.

---

3. Yummly

- **Overview:** Known for its smart personalization and integration with smart kitchen devices.
- **Strengths:** Suggests recipes based on user preferences, dietary needs, and taste.
- **Limitations:** Often depends on user preferences collected over time and not real-time ingredient input. Requires account setup and data tracking for optimal suggestions.

Identified Gaps in Existing Systems:

Limitation	Description
Time-Consuming	Users need to scroll through many recipes that may not match their available ingredients.
No Decision Intelligence	Systems don't consider ingredient combinations and their similarity to existing recipes.
No Real-Time Input Processing	Users can't just "type what they have" and get personalized results instantly.
Limited AI Usage	Most platforms rely on manual search or basic filtering — not ML-based recommendation.

Dishcovery: Bridging the Gap

Dishcovery is designed to overcome these challenges. Unlike traditional platforms, our system:

- Accepts real-time input of available ingredients.
- Uses ML (KNN + CountVectorizer) to find best-fit recipes based on ingredient similarity.
- Provides quick, ranked results with step-by-step cooking instructions.
- Allows searching and filtering to enhance usability.

Thus, Dishcovery represents a modern, intelligent step toward **personalized, efficient cooking** using AI.

PROPOSED SOLUTION

To address the limitations of existing recipe platforms and the common problem of "what to cook with what I have," we propose **Dishcovery – Your Cooking Assistant**, an intelligent and user-centric recipe recommendation system powered by **Machine Learning** and **modern web technologies**.

Key Features of the Proposed Solution

1. **Ingredient-Based Recipe Suggestions:**
    - Users can input the ingredients they currently have in their kitchen.
    - These inputs are vectorized using the **CountVectorizer** technique, converting textual data into a numerical format.
    - The system uses **K-Nearest Neighbors (KNN)** to compare user inputs against a database of recipes and recommend the closest matches.
2. **Personalized and Instant Recommendations:**
    - Recipes are sorted and ranked based on similarity to the user's input.
    - Suggestions are fast (under 2 seconds) and tailored to user needs.
3. **Detailed Recipe Cards:**
    - Each suggested recipe comes with:
        - Step-by-step cooking instructions
        - Dish type and preparation time
4. **Search and Filtering Capabilities:**
    - Users can search for specific recipe names or browse recipes by dish type
5. **Interactive User Interface:**
    - Built using **HTML, CSS, and JavaScript**, the frontend is clean, responsive, and beginner-friendly.
    - Users can navigate through recipe suggestions and select their desired dish with ease.
6. **Backend Integration:**
    - The backend is powered by **Node.js and Express.js**.
    - It handles API calls, recipe fetching, and ML model integration.
    - All recipes and user activity data are stored in a **MongoDB** database.

7. **Admin Dashboard and Analytics:**
    - Admins can log in to update recipe content and view analytics.
    - Charts and insights are generated using **Pandas**, **Matplotlib**, and **Seaborn**

Why This Solution Works

Unlike traditional platforms, Dishcovery's use of **AI for intelligent matching** transforms the user experience by:

- Reducing time spent on browsing irrelevant recipes
- Suggesting meals users can actually cook *right now*
- Encouraging culinary exploration with helpful tips and variations

Example Use Case

User Input: Tomato, Onion, Green Chili, Bread
Suggested Recipes:

- Bombay Masala Toast
- Bombay Sandwich
- Chana Chaat

In each case, the system evaluates the user's input and finds the closest matching recipes by analyzing the ingredients' text data, making the process seamless and helpful.

Dishcovery is not just a cooking app—it's a **smart kitchen assistant** that combines artificial intelligence with everyday cooking needs, making the act of preparing meals simple, efficient, and fun.

SYSTEM ARCHITECTURE

The architecture of **Dishcovery – Your Cooking Assistant** is modular, scalable, and built to ensure seamless communication between the frontend, backend, database, and the machine learning model. This system follows the classic **client-server model** with clear separation of concerns, making it easy to maintain and expand.

Architecture Overview

The system is divided into five major components:

1. Frontend (Client Side)
2. Backend (Server Side)
3. Machine Learning Module
4. Database
5. Admin & Analytics Dashboard

1. Frontend

- **Technologies Used:** HTML, CSS, JavaScript
- **Purpose:** Handles user interaction and user interface
- **Features:**
    - Accepts ingredient inputs
    - Displays suggested recipes as cards
    - Supports search and filtering functionality
    - Fully responsive across devices

2. Backend

- **Technologies Used:** Node.js, Express.js
- **Purpose:** Manages server-side logic, handles API requests and connects frontend with the ML model
- **Features:**
    - Accepts input data from the frontend
    - Passes ingredient data to the ML model

- Fetches recipe data from MongoDB
- Sends structured response back to frontend

3. Machine Learning Module

- **Technologies Used:** Python, scikit-learn, CountVectorizer, KNN
- **Purpose:** Implements the core logic for recipe recommendation
- **Functionality:**
  - Uses **CountVectorizer** to transform textual ingredients into vector form
  - Applies **K-Nearest Neighbors (KNN)** to match user inputs with similar recipes
  - Returns top relevant recipe indices to the backend

4. Database

- **Technology Used:** MongoDB
- **Purpose:** Stores all persistent data including:
  - Recipe details (name, ingredients, steps, tips, image URLs, etc.)
  - Admin credentials and analytics logs
- **Features:**
  - Schema-less flexibility for dynamic fields
  - Easily integrated with the backend for efficient CRUD operations

5. Admin & Analytics Dashboard

- **Technologies Used:** Python, Pandas, Matplotlib, Seaborn
- **Purpose:** Allows admins to:
  - View charts related to popular ingredients, frequently searched recipes
  - Analyze user behavior and system performance
  - Update or delete recipes as needed

With this architecture, Dishcovery offers a smooth, intelligent experience for users and a powerful backend system for developers and admins. The modularity also allows easy addition of new features in the future like voice assistant, meal planner, or user accounts.

TECHNOLOGY STACK

The development of **Dishcovery – Your Cooking Assistant** leverages a carefully selected combination of technologies, tools, and frameworks that support scalability, performance, and ease of integration between components. Each layer of the application is built using modern and reliable technologies to ensure a seamless user experience and maintainable architecture.

1. Frontend

Technology	Purpose
HTML	Defines the structure of web pages and forms for user input.
CSS	Styles the frontend with responsive layouts and visual appeal.
JavaScript	Adds interactivity, such as dynamically displaying recipe cards.

2. Backend

Technology	Purpose
Node.js	A JavaScript runtime used to build the server-side logic of the application.
Express.js	A lightweight web application framework for building APIs and handling routes.

3. Machine Learning / AI Layer

Technology	Purpose
Python	Core language for ML development and preprocessing.
scikit-learn	Implements algorithms like CountVectorizer and K-Nearest Neighbors (KNN).

4. Database

Technology	Purpose
MongoDB	A NoSQL document-based database to store recipe data and user/admin records.

Technology	Purpose
Mongoose	ODM library used with Node.js to define schema and handle database operations.

5. Data Visualization & Analytics

Technology	Purpose
Pandas	Data manipulation for backend analytics.
Matplotlib	Visualizing recipe and usage data in chart form.
Seaborn	Used to create aesthetically pleasing statistical plots.

Other Tools and Platforms

Tool / Platform	Purpose
VS Code	Code editor for frontend/backend/ML code.
Postman	Testing backend API endpoints.
MongoDB Compass	Cloud-based NoSQL database for storing recipes, easily integrates with Node.js

FUNCTIONAL WORKFLOW

The workflow of **Dishcovery – Your Cooking Assistant** is designed to be simple, user-friendly, and efficient. It ensures that users can input ingredients, receive personalized recipe suggestions, and explore cooking options with minimal effort.

The functional workflow is divided into **three major lanes**:

1. User Interaction & Recipe Recommendation

This is the core functionality that allows users to input ingredients and receive suggestions.

Steps:

1. The user visits the Dishcovery website.
2. Enters available ingredients into the input field
3. The frontend sends this data to the backend via an API call.
4. The backend passes the data to the ML module (Python-based).
5. Ingredients are vectorized using **CountVectorizer**.
6. The **KNN model** searches for recipes with similar ingredients.
7. The top matches are returned to the backend.
8. Backend fetches full recipe details from MongoDB Atlas.
9. Suggested recipes are sent to the frontend and displayed as interactive recipe cards.

2. Browsing and Filtering Recipes

In addition to ingredient-based recommendations, users can explore recipes manually.

Features:

- Search by Name: Users can look up a specific dish.
- Filter by Dish Type: Filter recipes like breakfast, snack, dessert, etc.
- View Recipe Cards: Each card includes name, image, ingredients, steps, and tips.

3. Dashboard Analytics

- Login securely
- View charts and analytics built using Pandas, Matplotlib, and Seaborn
    - e.g., most searched dishes, trending ingredients

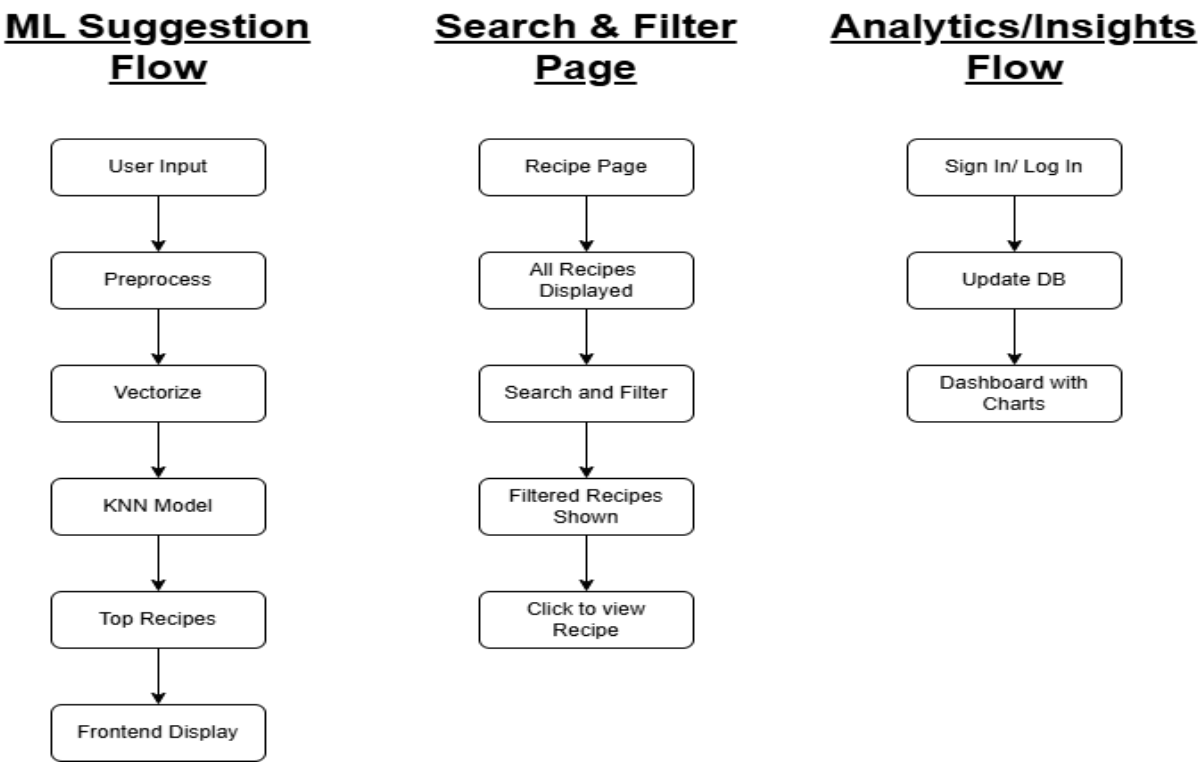


Fig : Functional Workflow Diagram of Dishcovery

This functional workflow ensures a seamless user experience while maintaining powerful backend intelligence and flexibility for future enhancements.

MACHINE LEARNING IMPLEMENTATION

The core of **Dishcovery – Your Cooking Assistant** is its ability to intelligently recommend recipes based on the ingredients a user has on hand. This is made possible through the use of the **K-Nearest Neighbors (KNN)** algorithm.

The model is trained to match a user's input (a list of ingredients) with recipes stored in a MongoDB Atlas database, using ingredient similarity as the matching criterion.

Model Overview

- **Goal:** Recommend recipes most relevant to the ingredients entered by the user.
- **Technique:**
  - Convert text-based ingredient data into numeric vectors using **CountVectorizer**
  - Find similar recipes using **K-Nearest Neighbors (KNN)** with **cosine similarity**
- **Tools Used:** Python, scikit-learn, MongoDB Atlas

Step 1: Connecting to MongoDB Atlas

The application fetches recipe data stored on **MongoDB Atlas**. The database contains documents with fields such as Recipe Name and Ingredients. Only necessary fields are retrieved to keep memory usage efficient.

```
load_dotenv()
MONGO_URI = os.getenv("MONGO_URI")

client = pymongo.MongoClient(MONGO_URI)
db = client["Recipe"]
collection = db["Full_Recipe"]

recipes = list(collection.find({}, {"_id": 0, "Recipe Name": 1, "INGREDIENTS": 1}))
```

Fig : Connecting to MongoDB Atlas and retrieving recipe data

Step 2: Preprocessing Ingredients

To ensure better model accuracy, ingredient text is cleaned:

- Converted to lowercase
- Extra spaces removed
- Common words like "salt" and "water" are ignored

```python
ignore = ['salt', 'water']

def process(ingredients):
    return ', '.join([
        ing.strip().lower()
        for ing in ingredients.split(", ")
        if ing.strip().lower() not in ignore
    ])
```

Fig: Preprocessing ingredients to remove noise and standardize input

---

Step 3: Vectorization Using CountVectorizer

The processed ingredients are converted into a numerical matrix using **CountVectorizer**. Each row in the matrix represents a recipe, and each column corresponds to an ingredient token.

```python
vectorizer = CountVectorizer(tokenizer=lambda x: x.split(', '))
matrix = vectorizer.fit_transform(cleaned_ingredients)
```

Fig: Ingredient text converted into vectors for similarity comparison

---

Step 4: KNN Model Training

The model is trained using the **K-Nearest Neighbors** algorithm with cosine similarity. It helps identify which recipes share the most ingredients with the user's input.

```python
model = NearestNeighbors(n_neighbors=10, metric='cosine')
model.fit(matrix)
```

Fig: Training the KNN model to identify similar recipes

Step 5: Generating Recipe Recommendations

When a user enters ingredients on the web app, those are passed to the Python script through command-line arguments. The same preprocessing and vectorization steps are applied, and the model returns the top matching recipe indices.

```python
user_input = sys.argv[1].lower().split(",")

user_vector = vectorizer.transform([', '.join(user_input)])
distances, indices = model.kneighbors(user_vector)

recommended_recipes = [recipe_names[i] for i in indices[0]]

print(json.dumps(recommended_recipes))
```

Fig: Generating top recipe recommendations from user input

Full Code:

```python
import pymongo
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.neighbors import NearestNeighbors
import json
import sys
from dotenv import load_dotenv
import os

load_dotenv()
MONGO_URI = os.getenv("MONGO_URI")
client = pymongo.MongoClient(MONGO_URI)
db = client["Recipe"]
collection = db["Full_Recipe"]

recipes = list(collection.find({}, {"_id": 0, "Recipe Name": 1, "INGREDIENTS": 1}))
ignore = ['salt', 'water']

def process(ingredients):
    return ', '.join([
        ing.strip().lower()
        for ing in ingredients.split(", ")
        if ing.strip().lower() not in ignore
    ])
recipe_names = [recipe["Recipe Name"] for recipe in recipes]
cleaned_ingredients = [process(recipe["INGREDIENTS"]) for recipe in recipes]

vectorizer = CountVectorizer(tokenizer=lambda x: x.split(', '))
matrix = vectorizer.fit_transform(cleaned_ingredients)

model = NearestNeighbors(n_neighbors=10, metric='cosine')
model.fit(matrix)

user_input = sys.argv[1].lower().split(",")
user_vector = vectorizer.transform([', '.join(user_input)])
distances, indices = model.kneighbors(user_vector)
recommended_recipes = [recipe_names[i] for i in indices[0]]
print(json.dumps(recommended_recipes))
```

.

This machine learning workflow successfully bridges everyday cooking challenges with AI-driven solutions, offering real-time, personalized, and meaningful recipe suggestions.

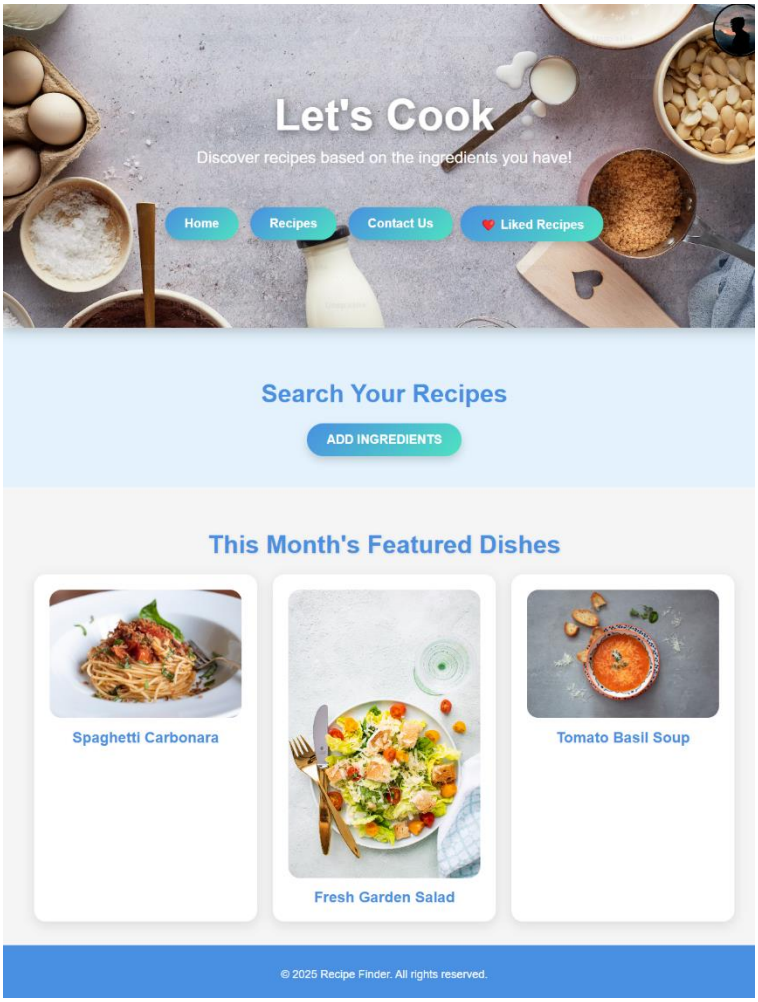
FRONTEND DESIGN

The frontend of **Dishcovery – Your Cooking Assistant** serves as the primary interface through which users interact with the system. It was built using **HTML**, **CSS**, and **JavaScript**, ensuring that the platform is clean, responsive, and intuitive for users of all cooking skill levels.

The user interface is structured to simplify the process of ingredient entry and recipe discovery. From entering ingredients to viewing personalized recipe suggestions, each part of the user journey is designed to be visually appealing and easy to navigate.

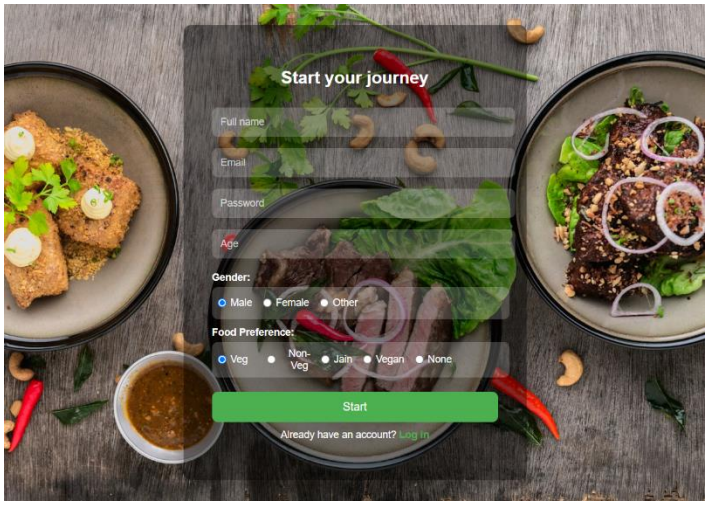
Home Page

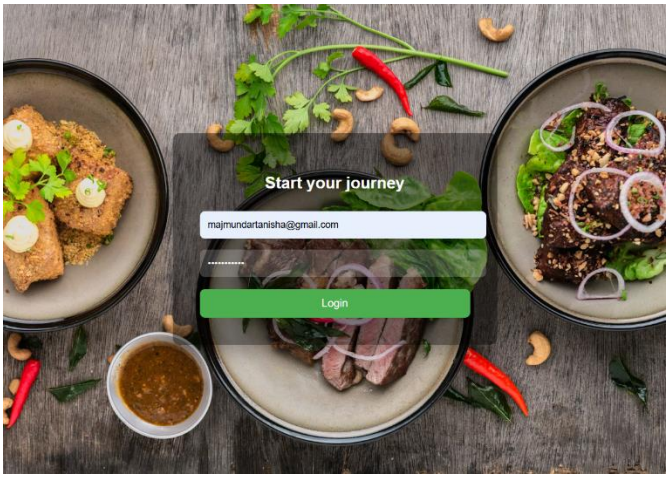
- **Description**: Acts as the landing page for users. Includes navigation links to "Login", "Signup", and possibly quick access to featured recipes.
- **Design**: Soft, inviting colors with rounded edges, consistent with the overall aesthetic.



User Signup & Login

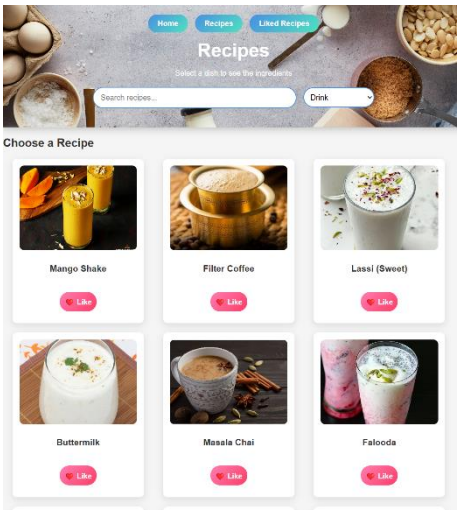
- **Functionality**:
    - **Signup Page**: Allows new users to register with fields for username, email, and password.
    - **Login Page**: Enables returning users to log in securely.
- **Design Consistency**: Both pages follow the same styling template with large input fields and pastel-colored buttons.





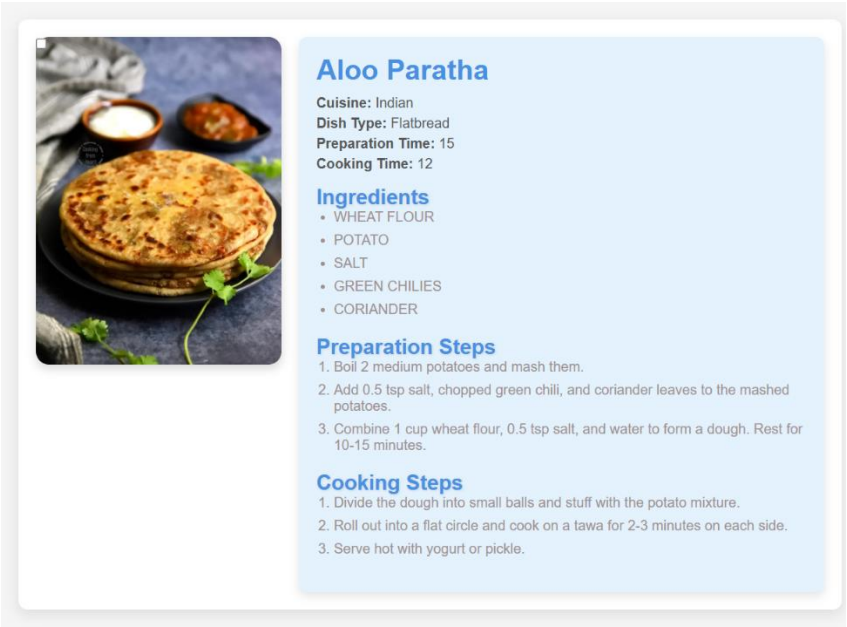
Recipe Discovery

- **Features**:
    - Display of multiple recipes with images, names, and basic descriptions.
    - Cards or tiles layout for clean browsing.
- **Search/Filter**: The page include search functionality to search recipes by names and filter functionality to filter recipes by dish type.



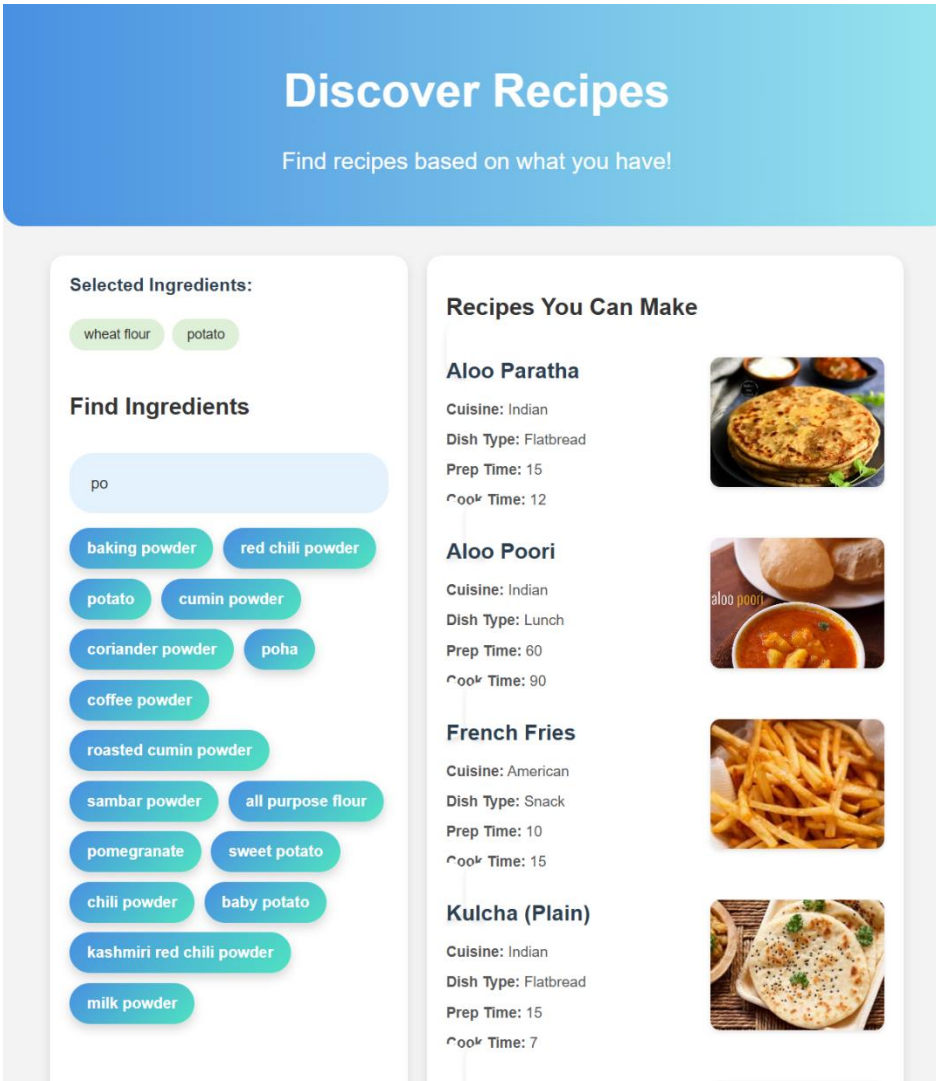
Recipe Detail View

- **Content Displayed**:
  - Full recipe name
  - Image
  - Cusine, Dish Type, Preparation Time, Cooking Time
  - Ingredients list
  - Step-by-step preparation instructions
  - Cooking Steps
- **Navigation**: Smooth transition from recipe list to detail view.



Ingredients Section

- **Functionality**:
  - Displays a categorized list or detailed view of ingredients used across recipes.
  - Selected ingredients are shown above
  - List of recipes that can be made by those ingredients are displayed on the right
  - May be integrated as a support page or as part of the recipe detail.

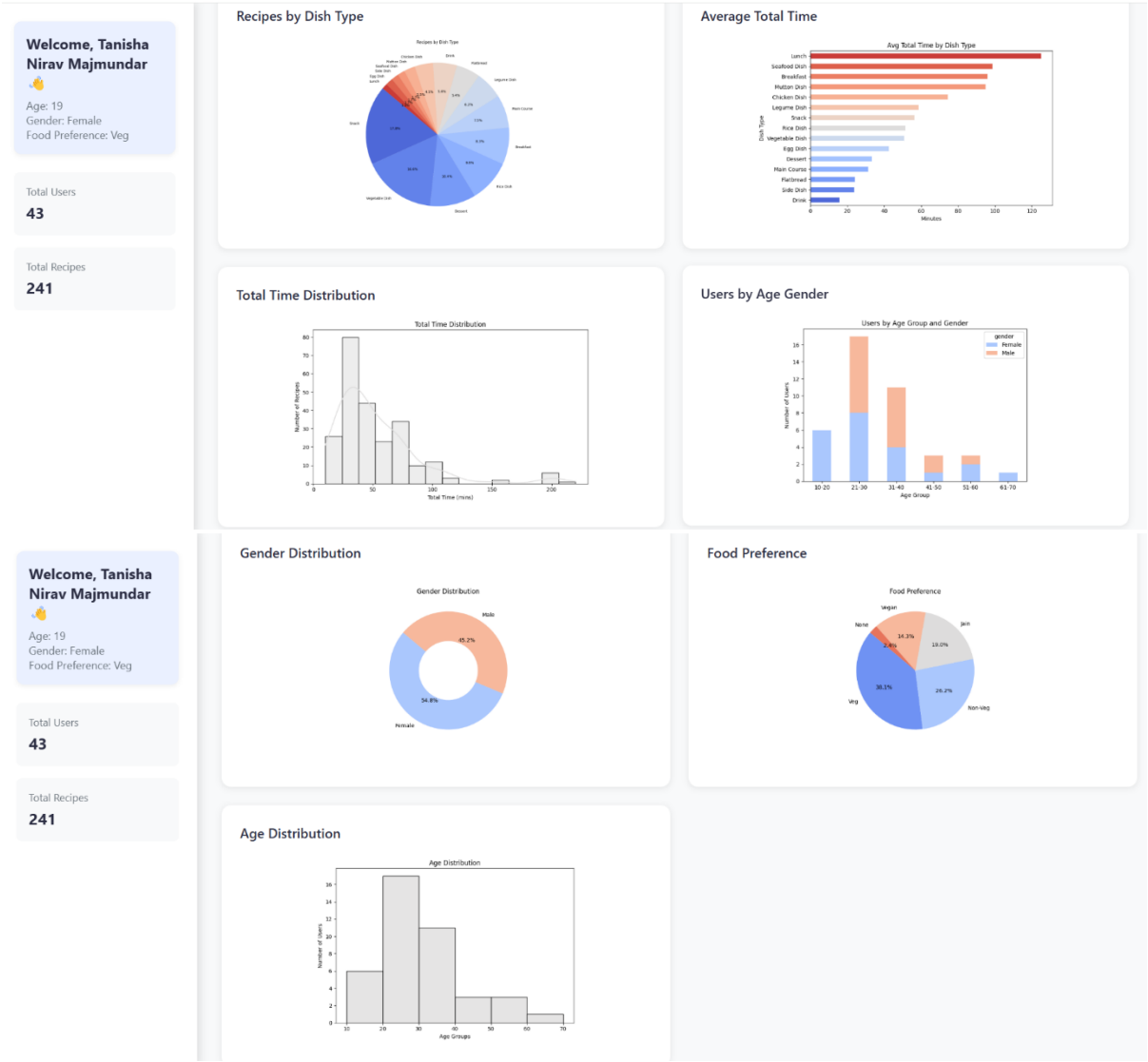


Responsive and User-Friendly Design

- All pages maintain:
  - Rounded corners
  - Consistent padding
  - Clear CTA (Call-to-Action) buttons
  - Light gradients and soft shadows

Data Analytics & Visualization

- **Purpose**: To extract insights from recipe usage, user preferences, and ingredient trends using data stored in MongoDB Atlas.
- **Implementation**:
  - Python script (recipe_charts.py) fetches data from MongoDB and generates charts.
  - Charts are saved under the charts/ directory.
- **Types of Charts Generated**:
  - **Top Ingredients**: Bar chart showing the most commonly used ingredients across all recipes.
  - **Recipe Category Distribution**: Pie or donut chart visualizing the proportion of recipes by category (e.g., breakfast, dinner, dessert).
  - **User Likes Analysis**: Bar chart showing most-liked recipes.
  - **Drink Pairing Frequency**: Visualization of popular drink matches across recipes.
- **Tools Used**:
  - pymongo for MongoDB data extraction
  - matplotlib and seaborn for plotting



Highlights of the Frontend Experience

- **Fast and interactive UI** with minimal load time
- **User-friendly form design** for ingredient input
- **Visually clear recipe cards** with essential details
- **Consistent color theme and fonts** for readability
- **Real-time feedback** on errors (e.g., empty input field)

BACKEND API IMPLEMENTATION

The backend of **Dishcovery – Your Cooking Assistant** is built using **Node.js** and **Express.js**, functioning as the core engine that powers communication between the frontend, machine learning module, and the cloud database (**MongoDB Atlas**).

The backend handles key responsibilities such as:

- User authentication
- Ingredient and recipe management
- Calling the machine learning model
- Delivering relevant data to the frontend via RESTful APIs
- Returning analytics and stats for the admin dashboard

API Routes

1. GET /all-recipes

- **Purpose:** Fetches a list of all available recipes from the MongoDB Atlas collection.
- **Usage:** Display recipes on “Explore Recipes” section.

2. POST /auth/login

- **Purpose:** Authenticates admin users.
- **Usage:** Used for secure login.

3. GET /details/:id

- **Purpose:** Fetches detailed information for a specific recipe by its ID.
- **Response includes:**
    - Ingredients
    - Steps
    - Tips/variations
    - Image URL
    - Cooking time

4. GET /ingredients

- **Purpose:** Returns a list of unique ingredients from all recipes.
- **Usage:** Useful for auto-suggestions, filters, or analytics.

5. POST /recommend

- **Purpose:** Accepts user-input ingredients and returns the top recommended recipes.
- **Process:**
    - Sends input to the **Python ML model**
    - Gets top N similar recipes based on KNN
    - Returns recipe names or IDs for rendering

6. GET /stats

- **Purpose:** Returns analytics data such as:
    - Total Number of user
    - Total Number ot Recipes
- **Usage:** Visualized on the dashboard using charts.

3. GET /recipes/

- **Purpose:** Fetches detailed information for a specific recipe by its ID or filters all the recipes based on dish type.

server.js (Main Server Entry Point)

- Initializes the Express application
- Loads environment variables
- Connects to MongoDB Atlas
- Applies global middleware (e.g., JSON parsing, CORS)
- Mounts all route modules (/recipe, /auth, /recommend, etc.)
- Starts the server on the configured port

Tech Stack Used

Component	Tool/Library
Web Framework	Express.js
Authentication	JSON Web Token (JWT) or custom session logic
DB Driver	Mongoose (for MongoDB Atlas)
ML Integration	child_process.spawn() to run Python
Data Format	JSON-based request/response

Security & Structure

- Admin routes (auth, stats) are protected
- Input validation prevents injection/malformed requests

DATABASE DESIGN

The database for **Dishcovery – Your Cooking Assistant** is implemented using **MongoDB Atlas**, a cloud-hosted NoSQL database service. MongoDB's document-oriented structure offers flexibility and scalability, making it ideal for storing diverse recipe data.

Each recipe is stored as a JSON-like document, allowing the application to quickly retrieve, filter, and search for relevant recipes based on user inputs.

Main Collection: Full_Recipe

Each document in the Full_Recipe collection contains the following fields:

Field Name	Description
Recipe Name	Title of the dish
Cuisine	Cuisine of the dish
Ingredients	Comma-separated list of ingredients
Dish Type	Category of dish (e.g., Breakfast, Dessert)
Preparation Time	Estimated time to prepare
Cooking Time	Estimated time to cook (in minutes)
Preparation Steps	Ordered list of instructions for preparation
Cooking Steps	Ordered list of instructions for cooking
Total Time	Estimated Total time
Image URL	Link to image


```
_id: ObjectId('679376ba9366660ed694fe46')
Recipe Name : "Tandoori Roti"
Cusine : "Indian"
Dish Type : "Flatbread"
INGREDIENTS : "WHEAT FLOUR, YOGURT, BAKING POWDER, SALT, WATER"
Preparation Time : 15
Cooking Time : 8
Preparation Steps : "1. In a large mixing bowl, combine 1 cup wheat flour, 0.5 tsp salt, an…"
Cooking Steps : "1. Preheat the tandoor or oven to 250°C (482°F). 2. Divide the dough i…"
Img URL : "https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcTsvL085SjADnnuvk…"
Total Time : 23
```

Users Collection

This collection is used for managing admin access to the dashboard and securing certain backend routes.

Field Name	Description
Name	Admin name
Email	Admin Email
password	Hashed password for authentication
Age	Age of the user
Gender	Gender of the user
Food	Food preference of the user


```
_id: ObjectId('67f1449c66dc2e2ce9df4041')
name : "Tanisha Majmundar"
email : "tanisha@example.com"
password : "$2b$10$YHhVRSgDTWm31ecigy2gnepmTcM3lesOHo2c2PLPJaQZ.qu0YrK82"
age : 19
gender : "Female"
food : "Veg"
__v : 0
```

Why MongoDB Atlas?

- **Schema Flexibility:** Easily adapt to new fields like nutrition, ratings, or user uploads.
- **Cloud Hosting:** Enables access from anywhere with secure connection.
- **Easy Integration:** Works seamlessly with Node.js backend and Mongoose ODM.

The use of MongoDB Atlas ensures that Dishcovery remains scalable, flexible, and efficient — especially when handling large datasets and real-time queries.

ADMIN DASHBOARD & ANALYTICS

The Admin Dashboard of **Dishcovery** provides powerful insights into recipe usage and user behavior. It is built using **Python**, with data fetched from **MongoDB Atlas** and visualized using libraries such as **Pandas**, **Matplotlib**, and **Seaborn**.

Two main analytics scripts are used:

- recipe_charts.py – Visualizes recipe data trends
- user_charts.py – Analyzes demographic and food preference data

All charts are automatically saved in the project's /frontend/charts directory and can be viewed by the admin.

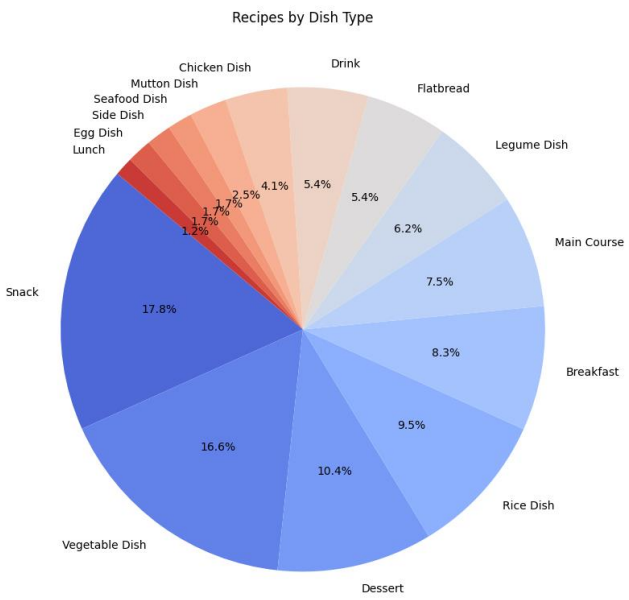
A. Recipe-Based Analytics (recipe_charts.py)

This script connects to the Full_Recipe collection and generates visual insights such as:

---

1. Recipe Count by Dish Type

- Pie chart showing how recipes are distributed across categories (e.g., breakfast, dessert)

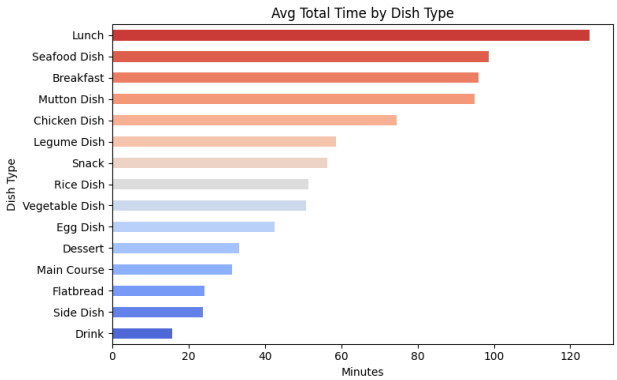



```python
user_input = sys.argv[1].lower().split(",")

user_vector = vectorizer.transform([', '.join(user_input)])
distances, indices = model.kneighbors(user_vector)

recommended_recipes = [recipe_names[i] for i in indices[0]]

print(json.dumps(recommended_recipes))
```

---

2. Average Total Time by Dish Type

- Bar chart showing average cooking time grouped by dish type ish type

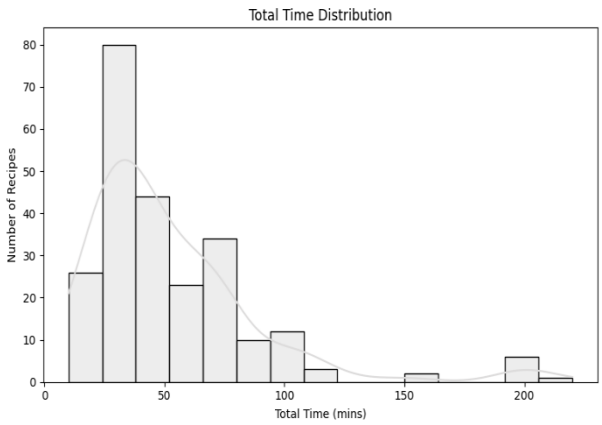



```
avg_prep_time = recipes.groupby('Dish Type')['Total Time'].mean().sort_values()
plt.figure(figsize=(8, 5))
colors = get_coolwarm_colors(len(avg_prep_time))
avg_prep_time.plot(kind='barh', color=colors)
plt.title('Avg Total Time by Dish Type')
plt.xlabel('Minutes')
plt.tight_layout()
plt.savefig(os.path.join(charts_path, "avg_total_time_by_dishtype.png"))
plt.show()
```

---

3. Total Time Distribution

- Histogram showing how recipes vary in total time required




```
plt.figure(figsize=(8, 5))
sns.histplot(recipes['Total Time'], bins=15, kde=True, color=get_coolwarm_colors(1)[0])
plt.title('Total Time Distribution')
plt.xlabel('Total Time (mins)')
plt.ylabel('Number of Recipes')
plt.tight_layout()
plt.savefig(os.path.join(charts_path, "total_time_distribution.png"))
plt.show()
```

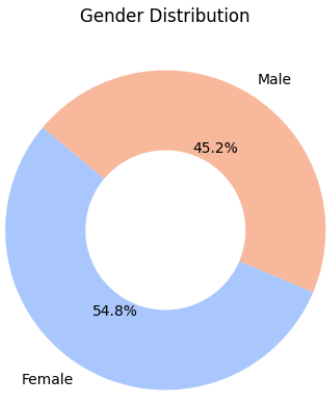
B. User-Based Analytics (user_charts.py)

This script uses the users collection and analyzes demographics and food preferences:

---

1. Gender Distribution

- Donut-style pie chart showing the ratio of male vs. female users

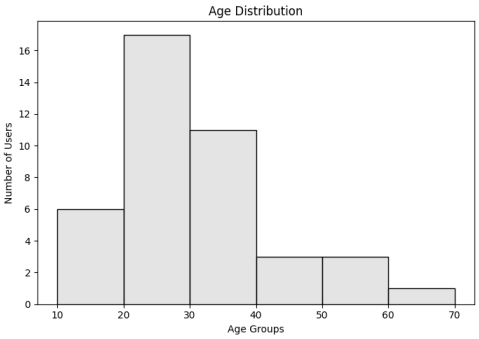



```
gender_counts = users['gender'].value_counts()
plt.figure(figsize=(5, 5))
colors = get_coolwarm_colors(len(gender_counts))
plt.pie(gender_counts, labels=gender_counts.index, autopct='%1.1f%%', startangle=140,
        wedgeprops={'width': 0.5}, colors=colors)
plt.title('Gender Distribution')
plt.savefig(os.path.join(charts_path, "gender_distribution.png"))
plt.show()
```

---

2. Age Distribution

- Histogram showing how users are spread across age groups

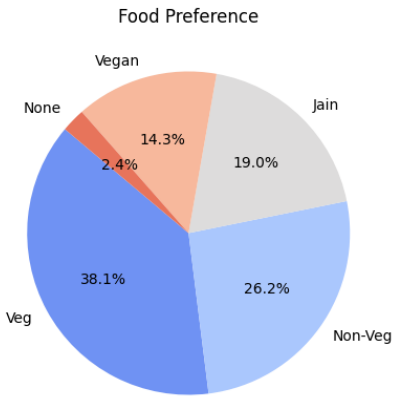



```
plt.figure(figsize=(7, 5))
sns.histplot(users['age'], bins=[10, 20, 30, 40, 50, 60, 70], kde=False, color=get_coolwarm_colors(1)[0])
plt.title('Age Distribution')
plt.xlabel('Age Groups')
plt.ylabel('Number of Users')
plt.tight_layout()
plt.savefig(os.path.join(charts_path, "age_distribution.png"))
plt.show()
```

---

3. Food Preferences

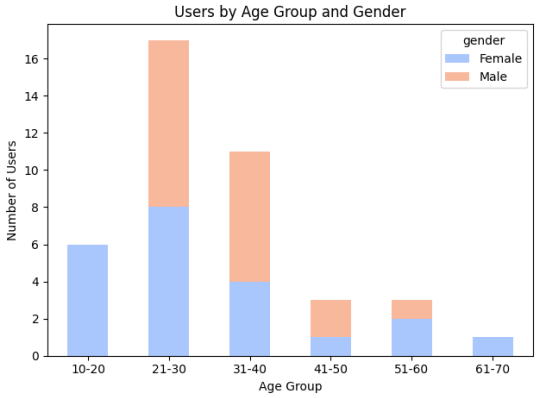
- Pie chart showing preferred diet types (e.g., vegetarian, non-vegetarian)




```
food_counts = users['food'].value_counts()
plt.figure(figsize=(5, 5))
colors = get_coolwarm_colors(len(food_counts))
plt.pie(food_counts, labels=food_counts.index, autopct='%1.1f%%', startangle=140, colors=colors)
plt.title('Food Preference')
plt.savefig(os.path.join(charts_path, "food_preference.png"))
plt.show()
```

4. Users by Age Group and Gender

- Stacked bar chart showing the gender breakdown across age ranges




```
bins = [10, 20, 30, 40, 50, 60, 70]
labels = ['10-20', '21-30', '31-40', '41-50', '51-60', '61-70']
users['AgeGroup'] = pd.cut(users['age'], bins=bins, labels=labels, right=False)
age_gender_data = users.groupby(['AgeGroup', 'gender']).size().unstack(fill_value=0)
colors = get_coolwarm_colors(len(age_gender_data.columns))
age_gender_data.plot(kind='bar', stacked=True, color=colors)
plt.title('Users by Age Group and Gender')
plt.xlabel('Age Group')
plt.ylabel('Number of Users')
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig(os.path.join(charts_path, "users_by_age_gender.png"))
plt.show()
```

Tools and Libraries Used

Tool	Role
Pandas	Data loading and grouping
Matplotlib	Static chart generation
Seaborn	Beautiful statistical visualizations
MongoDB Atlas	Cloud data source for both collections

Highlights

- All charts are saved automatically and ready to be displayed on the admin dashboard
- Easy to extend with filters or export options
- Helps admins understand platform growth and food trends

CHALLENGES FACED

During the development of **Dishcovery – Your Cooking Assistant**, the team encountered a variety of technical and non-technical challenges. These challenges were essential learning opportunities and helped strengthen both the project and the skills of the team members.

1. Integrating Machine Learning with the Backend

- **Challenge:** Connecting the Python-based ML model with the Node.js backend using child_process.
- **Resolution:** Used spawn() to run the ML script and capture its output. Debugging required careful formatting of inputs and handling JSON responses from Python.

2. Hosting MongoDB on the Cloud

- **Challenge:** Initially faced issues connecting securely to MongoDB Atlas, including connection timeouts and incorrect environment configurations.
- **Resolution:** Fixed using .env variables, whitelisted IPs, and tested credentials using MongoDB Compass before integration.

3. Designing a Clean Yet Functional UI

- **Challenge:** Balancing visual appeal with functional responsiveness across different devices.
- **Resolution:** Used a minimal and modular design with responsive CSS and tested layouts across screen sizes for mobile compatibility.

4. Generating Accurate Analytics from MongoDB

- **Challenge:** Aggregating data for charts and handling missing fields (e.g., missing age/gender in user entries).
- **Resolution:** Cleaned data using Pandas, applied default values where needed, and added field validation in MongoDB entries.

5. Data Quality & Ingredient Matching

- **Challenge:** Ingredient inconsistencies like “green chili” vs. “green chillies” affected ML accuracy.
- **Resolution:** Applied preprocessing and a custom ignore list to standardize common ingredient

CONCLUSION

Dishcovery – Your Cooking Assistant successfully delivers a smart, user-friendly solution to a very real and everyday challenge: deciding what to cook with limited ingredients. By combining **machine learning**, **web development**, and **data analytics**, the project not only suggests relevant recipes in real time but also enhances the overall user experience through search, filtering, and detailed recipe views.

Throughout the development of this project, the team gained hands-on experience in:

- Building a complete **MERN-stack web application**
- Integrating a **Python-based ML model** with a Node.js backend
- Using **MongoDB Atlas** for cloud-hosted, scalable data storage
- Creating dynamic and meaningful **data visualizations**
- Handling **real-world challenges** like inconsistent data, cross-language integration, and UI/UX design

The system is fast, scalable, and flexible — capable of expanding with features like **voice assistant support**, **user-uploaded recipes**, or **nutritional insights** in the future.

Overall, Dishcovery is a step toward smarter kitchens — where AI helps users save time, reduce food waste, and enjoy a personalized cooking experience.

FUTURE SCOPE

While **Dishcovery** already provides a smart and functional recipe recommendation system, there are several exciting features and improvements that can be incorporated in the future to make it even more intelligent, inclusive, and personalized.

Planned Enhancements

1. Voice Assistant Integration

- Allow users to speak ingredients instead of typing.
- Integrate with tools like Google Assistant or Alexa for a hands-free experience.

2. Weekly Meal Planner

- Generate personalized weekly meal schedules based on dietary preferences, time availability, and favorite cuisines.
- Helps users plan meals and grocery lists in advance.

3. User-Contributed Recipes

- Let users submit their own recipes to the platform.
- Include moderation and rating systems to ensure quality.

4. Nutritional Analysis

- Display calories, protein, fat, and carb information for each recipe.
- Suggest healthy alternatives for high-fat or high-sugar ingredients.

5. Multilingual Support

- Enable users to view recipes in multiple languages (e.g., Hindi, Gujarati, etc.)
- Makes the platform more accessible across regions.

Dishcovery has strong potential to evolve into a full-fledged **AI-powered cooking companion**, supporting sustainable cooking habits, dietary health, and smart kitchen experiences.